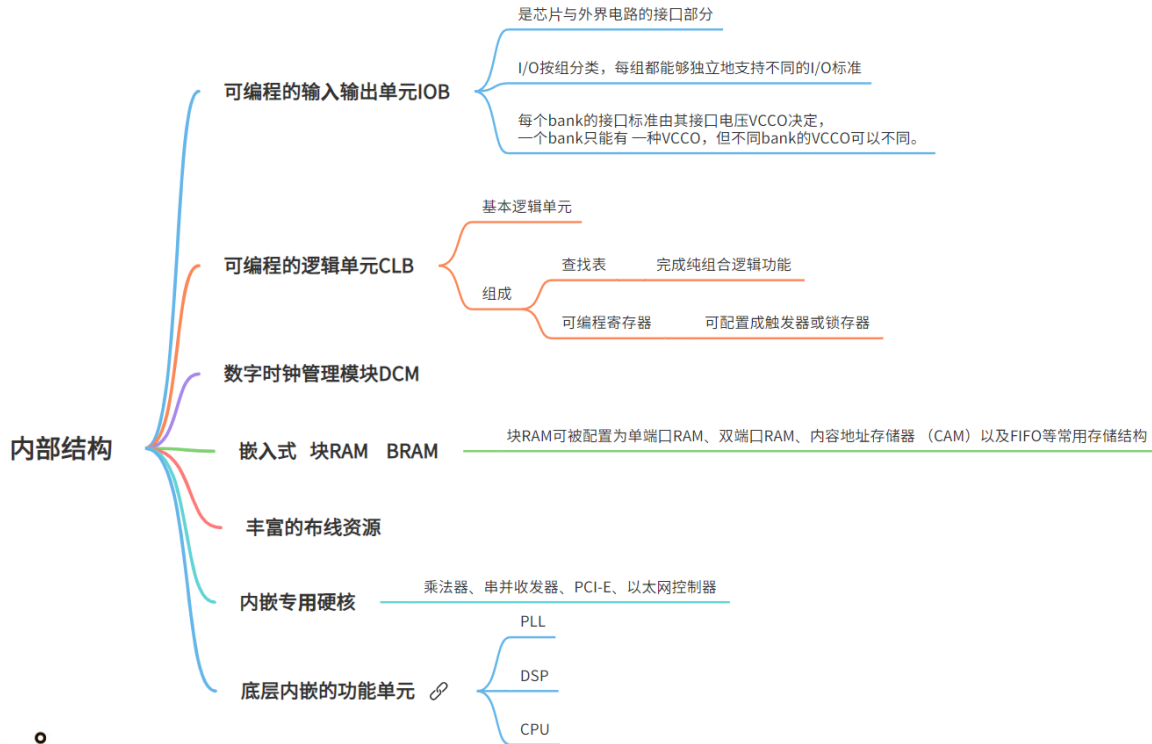


【卷一：FPGA相关内容】

1、FPGA 的内部结构



1.1 查找表

LUT里面存的什么：

- 逻辑表达式对应的真值表

如何使用LUT：

- 输入作为地址，去索引LUT里存的真值表内的对应结果
- 找到后将结果作为输出

理解：

- LUT是FPGA能“高速运算”的本质原因。因为它其实不是在运算，它是先把所有题目的答案都列成一个表存起来。等你出题的时候，它直接根据你的题目，直接找到答案给你。它根本没有算，它是准备好了非常非常多的“小抄”，看到题目直接写结果。

● 查找表(LUT)。

- 使用编程“表”（级联多路复用器）创建组合功能
- LUT输入是多路复用选择线

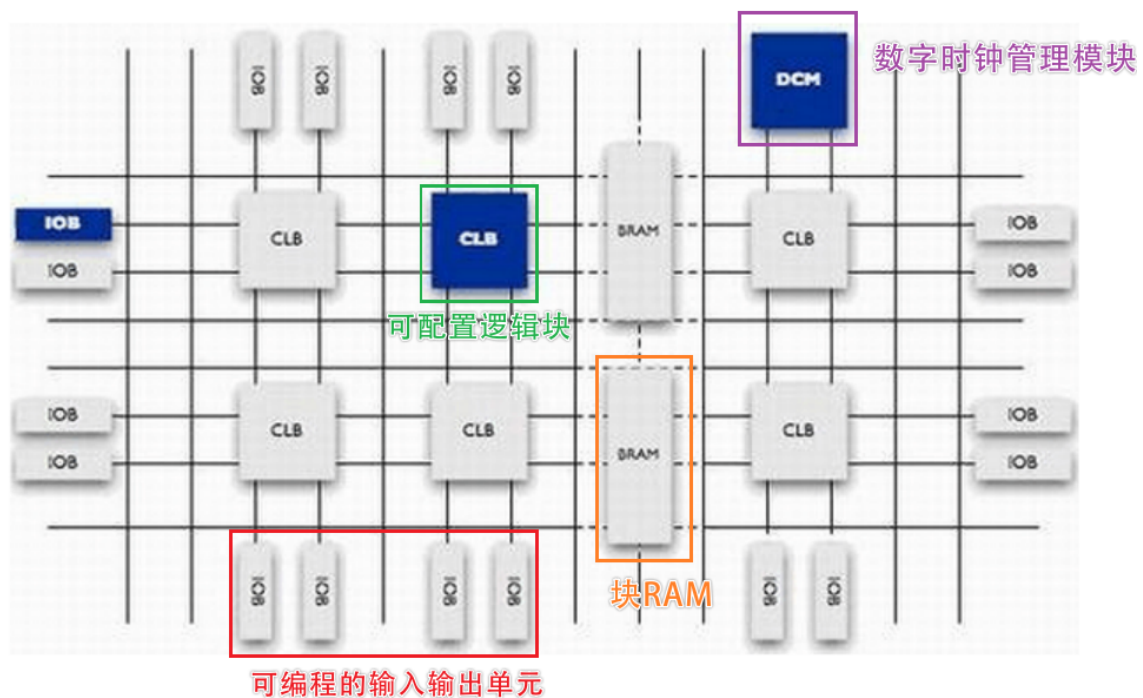
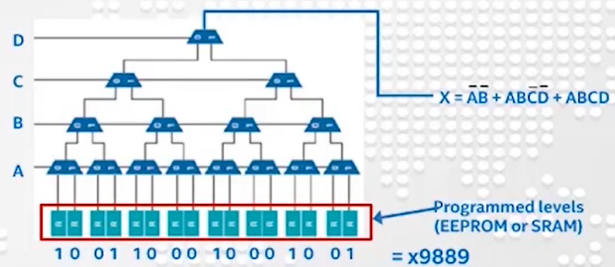


图 1-1 FPGA 芯片的内部结构

2、FPGA原理

FPGA 利用小型查找表（16×1RAM）来实现组合逻辑，每个查找表连接到一个 D 触发器的输入端，触发器再来驱动其他逻辑电路或驱动 I/O，由此构成了既可实现组合逻辑功能，又可实现时序逻辑功能的基本逻辑单元模块

3、FPGA的开发流程

3.1 英特尔-Quartus

1. **设计规范：**在 FPGA 设计项目开始之前，必须有系统功能的定义和模块的划分，另外就是要根据任务要求，如系统的功能和复杂度，对工作速度和器件本身的资源、成本、以及连线的可布性等方面进行权衡，选择合适的设计方案和合适的器件类型。
2. **设计输入：**指使用硬件描述语言将所设计的系统或电路用代码表述出来。最常用的硬件描述语言是 Verilog HDL。

3. **行为仿真/功能仿真**：指在逻辑综合之前对用户所设计的电路进行逻辑功能验证。仿真前，需要搭建好测试平台并准备好测试激励，仿真结果将会生成报告文件和输出信号波形，从中便可以观察各个节点信号的变化。如果发现错误，则返回设计修改逻辑设计。常用仿真工具有 Model Tech 公司的 ModelSim、Synopsys 公司的 VCS 等软件。
4. **综合仿真**：所谓综合就是将较高级抽象层次的描述转化成较低层次的描述。综合优化根据目标与要求优化所生成的逻辑连接，使层次设计平面化，供 FPGA 布局布线软件进行实现。就目前的层次来看，综合优化是指将设计输入编译成由与门、或门、非门、RAM、触发器等基本逻辑单元组成的逻辑连接网表，而并非真实的门级电路。真实具体的门级电路需要利用 FPGA 制造商的布局布线功能，根据综合后生成的标准门级结构网表来产生。为了能转换成标准的门级结构网表，HDL 程序的编写必须符合特定综合器所要求的风格。常用的综合工具有 Synplicity 公司的 Synplify、Synplify Pro 软件以及各个 FPGA 厂家自己推出的综合开发工具。
5. **布局布线**：可理解为利用实现工具把逻辑映射到目标器件结构的资源中，决定逻辑的最佳布局，选择逻辑与输入输出功能链接的布线通道进行连线，并产生相应文件（如配置文件与相关报告）；实现是将综合生成的逻辑网表配置到具体的 FPGA 芯片上。由于只有 FPGA 芯片生产商对芯片结构最为了解，所以布局布线必须选择芯片开发商提供的工具。
6. **门级仿真**：考虑延时
7. **编程调试**：设计的最后一步就是编程调试。芯片编程是指产生使用的数据文件（位数据流文件，Bitstream Generation），将编程数据加载到 FPGA 芯片中；之后便可进行上板测试。最后将 FPGA 文件（如 .bit 文件）从电脑下载到单板上的 FPGA 芯片中。

3.2 赛灵思-vivado

1. 系统规划：在FPGA设计项目开始之前，需要进行系统的功能定义和模块的划分。然后根据任务要求（系统的功能和复杂度），对工作速度和器件本身的资源，成本，以及连线的可布性进行评估，从而选择合适的设计方案和器件类型。
2. RTL输入：RTL, Register Transfer Level, 直译为寄存器转换级，要描述各级寄存器（时序逻辑中的寄存器），以及寄存器之间的信号的是如何转换的（时序逻辑中的组合逻辑）。通俗来讲，RTL 的输入一般为两种，使用硬件描述语言Verilog HDL/VHDL进行编写或者原理图输入。原理图就是比较老的做法了，通过门电路的拖拽连接起来设计系统，所以现在基本都是用语言来描述了。

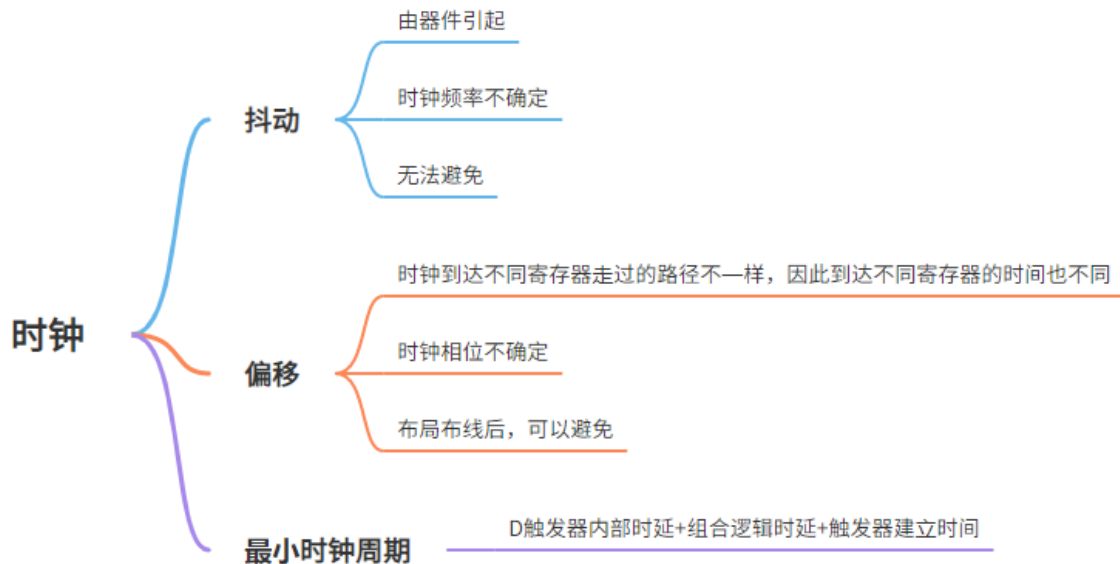
*我们虽然在system generator中是在拖拽连接，但实际上，在导出为网表文件时它已经转化成Verilog 语言描述了。也就是说，此时的RTL输入其实还是硬件描述语言输入而非原理图输入。

3. 行为仿真/功能仿真：在编译前对用户所设计的电路进行逻辑功能验证，此时是没有任何延迟信息的，仅对初步的功能进行检测。
4. 逻辑综合：综合的含义就是将高级层次的描述转化为低级层次的描述，就目前层次来看，综合优化是指将设计输入编译成基本逻辑单元组成的逻辑连接网表。
5. 综合后仿真（可选）：综合后仿真检查综合结果是否和原设计一致。仿真时，将综合生成的标准延时文件反标注到综合仿真模型中，可以估计门延时带来的影响，但是无法估计线延时，因此和布线后的实际情况有一定的差距。一般的设计可以省略这一步。
6. 综合后设计分析（时序及资源）：综合之后会告诉我们，目前的系统设计消耗了多少FPGA的资源，比如，消耗了多少LUT、RAM、DSP48,等等。我们可以根据这些报告来选择对设计进行优化。
7. 设计实现（包括布局布线及优化）：利用实现工具把逻辑映射到目标器件结构的资源中。布局将逻辑网表中的硬件原语和底层单元合理的配置到芯片内部的固有硬件结构上，需要在速度最优与面积最优之间做出选择；布线在布局的基础上，利用芯片内部的各种连线资源，合理正确的连接各个元件。
8. 布线后仿真：意思与前面的综合后仿真一致，因为此时已经进行了布局布线，所以在时序中包含的延迟信息更真实。能较好地反映芯片的实际工作情况。

9. 板级调试：产生使用的数据文件（bitstream-比特流文件），然后将编程数据下载到FPGA芯片中，测试实际运行结果。最好的方式是外接逻辑分析仪查看，但这样需要占用一些IO接口，而且一般人手头没有逻辑。那比较实用的方向就是使用内嵌式逻辑分析仪ILA。【详情可见本人视频，开发笔记第二期-开发流程】

10. bitstream固化：这其实是最最最最后一步了。只有在你确保当前的设计已经完美无瑕可以拿来用的时候，才会把它固化到FPGA上。这样，每次上电运行的就变成这个系统了。

4.时钟



5、毛刺信号

出现原因：

- 输入信号经过FPGA内部布线以后产生不同延时的异步电路
- 在 编程时内部信号变化落后

解决方法：

- 将信号经过两级触发器就可以消除毛刺

6、组合逻辑、时序逻辑概念

组合逻辑：

- 输入信号一旦发生改变，输出信号随之立马发生改变；且输出仅仅和输入有关
- `always@(*)`、`assign`

时序逻辑：

- 输出信号不仅和输入信号有关，还和时钟有关
- `always@(posedge clk)`

7、竞争冒险（组合逻辑中）

```
output = A && (~A)
```


竞争：由于不同路径上的门级数是不同的，因此信号经过不同路径的时间也不同，因此会导致传输时间不同，在信号变化的瞬间存在先后顺序。这种现象叫竞争

冒险：由于竞争而引起电路输出信号中出现了非预期信号，产生瞬间错误的现象称为冒险。表现为毛刺

如上，($\sim A$)比A多走了一个反向门，因此到达与门的时间要晚一点，这就会导致最后输出信号有一小段脉冲，这就是竞争冒险现象

存在：只要输出逻辑表达式中含有某个信号的原变量A和反变量A'之间的“与”或者“或”关系，且A和A'经过不同的传播路径，则存在竞争

解决方法：采样时序逻辑，仅在时钟边沿采样

将组合逻辑操作写入always@中，每个上升沿才执行一次逻辑操作，这样就消除竞争了

补充：

办法一：修改逻辑表达式避免以上情况【需要逻辑分析能力】

办法二：采样时序逻辑，仅在时钟边沿采样【推荐，事实上也最常用】

办法三：在芯片外部并联电容消除窄脉冲【物理方法】

8、关键路径

指同步逻辑电路中，组合逻辑**时延最大的路径**，也就是说关键路径是对设计性能起决定性影响的时序路径，组合逻辑的延时过长，就会成为关键路径

怎么找到关键路径：

- 静态时序分析能够找出逻辑电路的关键路径。

如何从关键路径入手，提升性能：

- 对关键路径进行时序优化，可以直接提高设计性能，可以使用流水线来缩短关键路径

9、建立时间、保持时间（D触发器中）

- 建立时间：触发器在时钟上升沿到来之前，输入信号必须保持稳定的时间
- 保持时间：触发器在时钟上升沿到来之后，输入信号必须保持稳定的时间

为什么需要建立时间和保持时间：

- 因为触发器内部数据形成需要一定的时间

如果不满足建立时间、保持时间会有什么后果：

- 触发器会进入亚稳态状态，最终的输出端会任意输出0、1

10、亚稳态

亚稳态出现在哪：

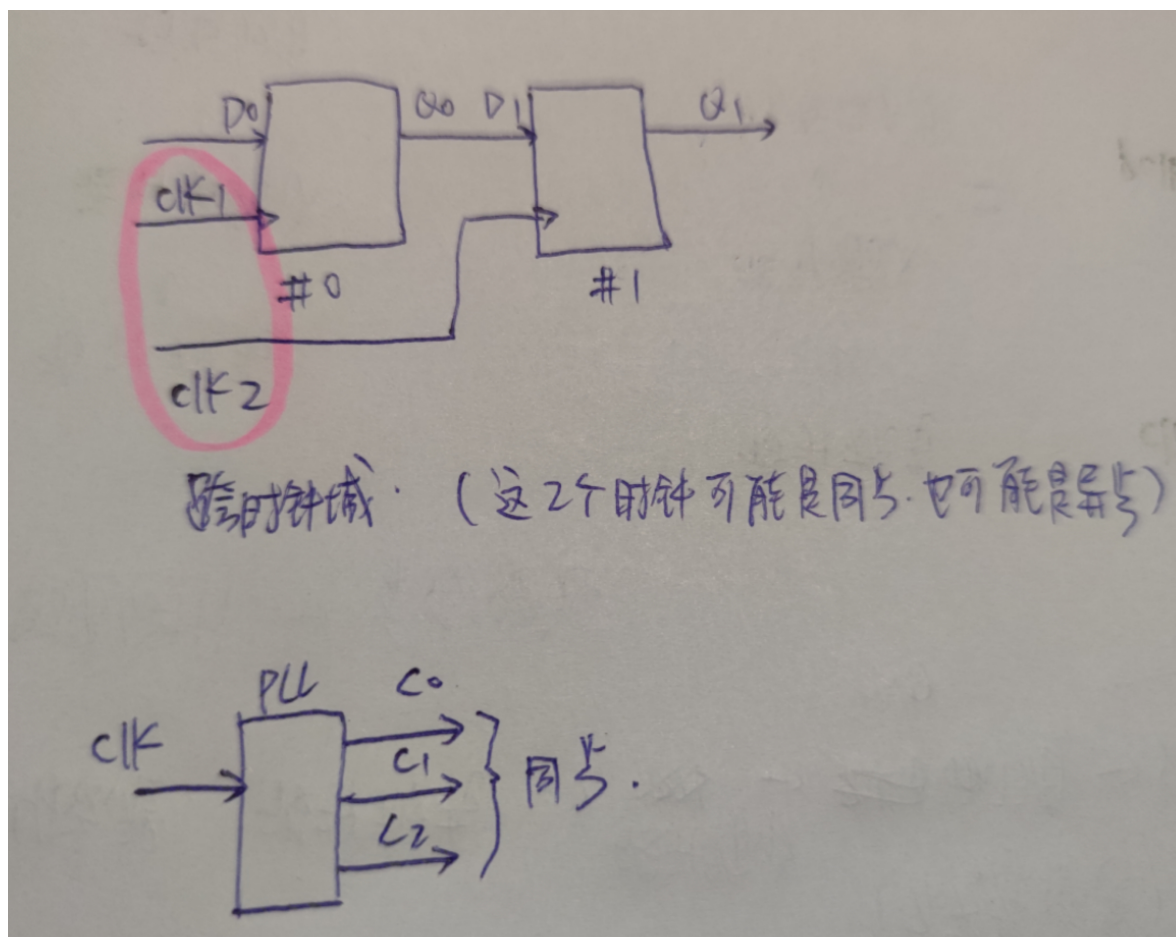
- 触发器的建立时间、保持时间不满足时，会发生亚稳态
- 异步电路（即输入信号是异步的）会出现亚稳态
- 跨时钟域会出现亚稳态

如何减小亚稳态发生的概率：

- 单比特信号：使用同步器进行同步（2级寄存器打拍）
- 多比特信号：FIFO
- 复位电路使用异步复位、同步释放

11、同步

11.1 同步时钟、异步时钟



11.2 同步电路、异步电路

同步电路：核心电路是时序逻辑

异步电路：核心电路是组合逻辑

11.3 同步复位、异步复位、异步复位同步释放

同步复位：

- always@(posedge clk)
- 输出信号和clk上升沿对齐

异步复位：

- always@(posedge clk or negedge rst_n)
- 输出信号和clk上升沿没有对齐

异步复位同步释放：

- 其根本思想，就是将异步信号同步化

```
module test(  
    input clk,  
    input rst_n,  
    input a,  
    output reg c  
);  
  
reg b,rst_nr;  
  
always@(posedge clk)  
  
    rst_nr <= rst_n; //关键点  
  
always@(posedge clk or negedge rst_nr)  
  
    if(!rst_nr) begin  
  
        b <= 1'b0;  
  
        c <= 1'b0;  
  
    end  
  
    else begin  
  
        b <= a;  
  
        c <= b;  
  
    end  
  
endmodule
```

下列哪种说法正确()。(ABD)

- A 同步复位增加了时序收敛的难度
- B 同步化异步复位可以保证逻辑正确复位
- C 从使用资源的角度看，应该使用异步复位
- D 异步复位可能会导致逻辑错误

分析：

Altera器件中的基本单元就是由异步复位且低有效结构的寄存器组成的，如果不按照这个现有的资源来设计我们就需要额外使用其他的资源，从而造成浪费，所以这就是我们所“默认”Altera器件中往往都是异步复位低有效的原因。

但Xilinx与之正好相反。

官方文档《Vivado使用误区与进阶》中明确说明尽量避免使用异步复位，且如果使用复位就用高复位，巧的是刚好和Altera推荐的规则相反，也是因为内部结构决定的。

关于这一点还真有很多分析：

[分析1](#)

[分析2](#)

总之结论就是，Xilinx FPGA的设计：

最好没有复位，有也要用同步高有效复位。

因此C选项对于Altera来说是正确的，对于Xilinx来说是错误的。

最终认为C选项错误。

11.4 同步FIFO、异步FIFO

时钟是否用了一个

12、FIFO

FIFO的宽度、深度

如何计算FIFO深度：

假设写时钟=50MHZ、读时钟=20MHZ

- 计算写入120个数据需要的时间： $120 \times 20\text{ns} = 2400\text{ns}$
- 在相同的时间里可以读取多少个数据： $2400 \times 10^{-9} \times 20 \times 10^6 = 48$
- FIFO 最小深度： $120 - 48 = 72$

12.1 FIFO最小深度计算

基本大原则：写入的数据量=读出的数据量

FIFO不能用在连续数据流的缓存中。

例如:AD采样的结果连续不断的送入FIFO，深度只有无穷才能满足需求。



在出现Burst的时候，凸显FIFO的功能与用途

Burst：是指在同一行中相邻的存储单元连续进行数据传输的方式，连续传输的周期数就是突发长度（Burst Lengths，简称BL）

发送方“突发”发送数据的时间T内，读不完所有的数据。
T时间后，FIFO内有足够的空间存储尚未被读走的数据，
在发送方不发数据的空闲时间内，将剩余的数据读走。

→ FIFO深度的设置

12.1.1写速度>读速度

fa>fb 基础情况

Case:fa=80MHz，fb=50Mhz，每一次burst传送120个数据

写入120数据所用时间：

$$120/fa=1500ns$$

1500ns可以读出的数据：

$$1500*fb=75$$

仍未被读出数据：

$$120-75=45 \text{ -----FIFO最少深度}$$



格雷码比较时Delay的存在

FIFO最少深度>45

12.1.2 写速度>读速度：读写 有空闲

idle cycle: 闲置循环

读写过程不是在每个周期都连续进行的, 可能会存在某些个闲置的循环。



计算时候相当于变相减小了频率

换个说法: wr_en / re_en 占空比
计算方法相同

fa>fb 读写存在idle cycle

Case: $fa=80\text{MHz}$, $fb=50\text{MHz}$, 每一次burst传送120个数据

两次写之间idle cycle=1
两次读之间idle cycle=3

写入一次数据时间= $2 \times (1/fa)$ ---40MHz(fa')

读出一次数据时间= $4 \times (1/fb)$ ----12.5MHz(fb')

写入120数据所用时间:

$120/fa'=3000\text{ns}$

3000ns可以读出的数据:

$3000 \times fb'=37.5$

仍未被读出数据:

$120-37.5=82.5 > 83$ -----FIFO最少深度

12.1.3 写速度<读速度: 读写有空闲

fa<fb 读写存在idle cycle



Case: $fa=50\text{MHz}$, $fb=60\text{MHz}$, 每一次burst传送120个数据

两次写之间idle cycle=1

两次读之间idle cycle=2

写入一次数据时间= $2 \times (1/fa)$ ---25MHz(fa')

读出一次数据时间= $3 \times (1/fb)$ ----20MHz(fb')



相当于fa<fb 计算方法同第一页

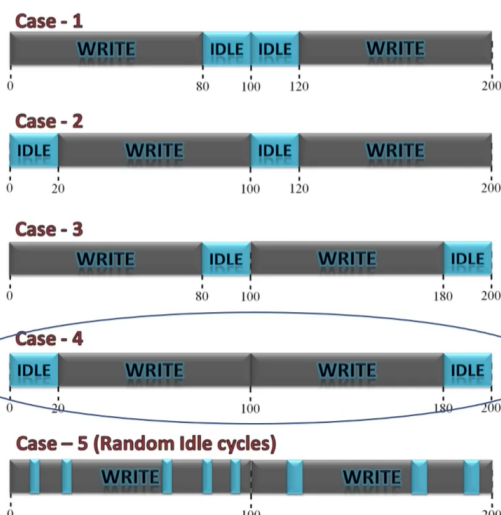
12.1.4 写速度<读速度: 随机读写

fa<fb 随机读写情况

Case: 写入数据= 80 DATA/100 Clock

读出数据= 8 DATA/10 Clock. Burst数据量160

写频率80MHz, 读频率50MHz



写入160数据所用时间:

$160/fa=2000\text{ns}$

2000ns可以读出的数据:

$fb'=(1/50\text{MHz}) \times 10/8=40\text{MHz}$

$2000\text{ns} \times fb'=80$

仍未被读出数据:

$160-80=80$ -----FIFO最少深度

背靠背

最差的情况: 写最快, 读最慢

13、跨时钟域

13.1 单比特

- 慢——快：同步器打2拍
- 快——慢：握手
 - 将输入数据展宽
 - 将展宽数据打2拍同步到慢时钟域
 - 再将打的第一拍得到的数据打2拍后将次上升沿
 - 得到输出数据
 - 将检测上升沿信号打2拍同步到慢时钟域下后，拉低展宽信号

```
module handshake_sync (
    input clk1 , //快时钟信号
    input sys_rst_n , //复位信号，低电平有效
    input read , //信号，快时钟域的
    input clk2 , //慢时钟信号

    output read_sync_pulse //输出信号
);

//in1表示该信号在clk1时钟域
reg req_in1 ;
reg ack_in1 ;
reg ack_in1_dly1 ;
//in2表示该信号在clk2时钟域
reg req_in2 ;
reg req_in2_dly1 ;
reg req_in2_dly2 ;
//*****
//**                               main code
//*****

//1、clk1时钟域下req信号的生成
always @(posedge clk1 or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0)
        req_in1 <= 1'b0;
    else if(read == 1'b1)
        req_in1 <= 1'b1;
    else if(ack_in1_dly1 == 1'b1)
        req_in1 <= 1'b0;
    else
        req_in1 <= 1'b0;
end

//2、clk2时钟域下req信号的采样
always @(posedge clk2 or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0) begin
        req_in2 <= 1'b0;
        req_in2_dly1 <= 1'b0;
        req_in2_dly2 <= 1'b0;
    end
    else begin
        req_in2 <= req_in1;
```



```

        req_in2_dly1 <= req_in2;
        req_in2_dly2 <= req_in2_dly1;
    end
end

//3、clk1时钟域下ack信号的采样 直接采样req_in2_dly1作为ack信号即可
//这是因为有了req_in2和req_in2_dly1之后我们就可以生成dout，所以此时就可以返回ack信号了
always @(posedge clk1 or negedge sys_rst_n) begin
    if (sys_rst_n == 1'b0) begin
        ack_in1 <= 1'b0;
        ack_in1_dly1 <= 1'b0;
    end

    else begin
        ack_in1 <= req_in2_dly1 ;
        ack_in1_dly1 <= ack_in1;
    end

end

end

//4、dout信号的产生

assign read_sync_pulse = req_in2_dly1 & ~req_in2_dly2;

endmodule

```

13.2 多比特

- 使用异步FIFO进行数据同步
- 握手进行数据同步

13.3 异步信号同步化

- 情况一：当不同模块使用了不同快慢的时钟时。如一个100M一个50M，快慢通道之间进行数据交流时需要考虑信号同步的情况。如果是快时钟通道获取慢时钟通道信号，方法比较简单。而慢通道获取快通道，那就得根据情况考虑了。
- 情况二：当FPGA和外部设备进行数据交换时。FPGA通常来讲都是速度比较快的那个，这种与其他设备进行数据交换通常都不会只是单bit位的数据交换，而是多位数据/总线数据交换。那么FIFO和DRAM的使用是必不可少的，如果是与ARM进行数据交换，那么如果有DMA那就最棒了。在进行ZYNQ系统设计时，PL端与PS端之间的数据交互，我通常都会使用到FIFO和DMA来实现数据的高速传输

14、RAM、ROM

RAM：

- 任何时候都可以读写
- 断电丢失
- 常称为内存，或移动内存

ROM：

- 只读存储器
- 它相当于PC机上的硬盘，用来存储和保存数据。
- ROM数据不能随意更新，但是在任何时候都可以读取。
- 即使是断电，ROM也能够保留数据
- 但是资料一但写入后只能用特殊方法或根本无法更改。现在，EEPROM出现了，它可以随意的修改rom中的内容了。
- flash属于广义的EEPROM

15、DMA

涉及到高速数据传输时，DMA就显得非常重要了。

从字面意思上看，DMA即为“直接内存读取”的意思，换句话说DMA就是用来传输数据的，它也属于一个外设。只是在传输数据时，无需占用CPU。

1. PS通过AXI-lite向AXI DMA发送指令，
2. AXI DMA通过HP通路和DDR交换数据，
3. PL通过AXI-S读写DMA的数据。

16、DDR3

[DDR3的带宽计算](#)

17、 [7:0] rom [255:0]

[每个寄存器单元的大小] rom [有多少个这样的]

- 该存储器有256个8位的存储器。
- 该存储器的地址范围是0到255

如果想对memory中的存储单元进行读写操作，必须指定该单元在存储器中的地址

reg[7:0] mema[255:0]正确的赋值是（A）

- A. mema[5]=3'd0
- B. mema[5][3:0]=4'd1
- C. mema[8][8] = 8'd0
- D. 都正确

题目中就是reg [7:0] mema[255: 0];

该存储器有256个8位的存储器。

A选项，mema[5]=3'd0，是对第5地址单元赋值，没有问题。

B选项，mema[5][3:0]=4'd1，是对第5地址单元的最低4位赋值，正确的数组赋值应该为 mema【address】 = data。

C选项，mema[8][8] = 8'd0，是对第8地址单元的第9位进行赋值，但这是个8位寄存器，所以应该是不对的。

正确的数组赋值应该为 `mema【address】 = data`。对于一个地址的数据的某一位或者几位赋值，应该添加一个中间变量；比如：`a = mema【address】`，`a【3:1】 = data`；某一地址对应的数据的某几位也是这种操作

18、奇偶分频

18.1 偶分频

- 计数器：计到N/2-1清零
- 当计数器计到最大值，输出取反

18.2 奇分频

- 计数器：计到N-1
- 时钟上升沿：计到 $(N-1)/2-1$ 拉高，计到N-1拉低
- 时钟下降沿：计到 $(N-1)/2-1$ 拉高，计到N-1拉低
- 最重要：输出=上升沿 && 下降沿

19、奇偶校验

使用单目缩位异或求出多位数据的0/1的个数

```
// 奇偶校验
`timescale 1ns/1ns
module odd_sel(
    input [31:0] bus,
    input sel,
    output check
);
//*****code*****//

wire even_1;// 偶校验
wire odd_0;// 奇校验

assign odd_0 = ^bus;
assign even_1 = ~odd_0;

assign check = (sel)?odd_0:even_1;

//*****code*****//
endmodule
```

VL3 奇偶校验

[题目](#)[题解\(27\)](#)[讨论\(22\)](#)[排行](#)

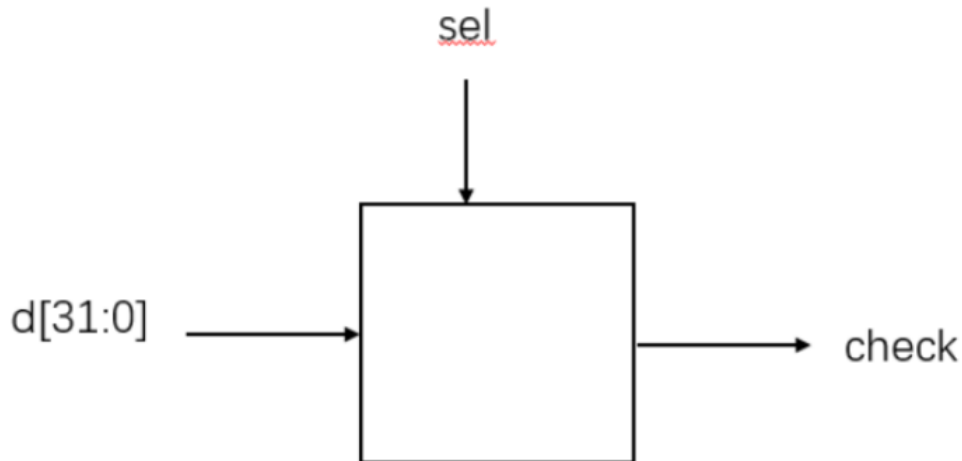
简单  通过率: 42.57%

描述

题目描述:

现在需要对输入的32位数据进行奇偶校验,根据sel输出校验结果 (1输出奇校验, 0输出偶校验)

信号示意图:



【卷二：Verilog相关内容】

1、可综合的变量类型

reg、wire、integer

HDL的变量类型:

Buffer: 这个上期有讲到, 应该是VHDL的端口类型。

String: 在VHDL中, string是 character 类型的一个非限定数组。

Double: 双精度浮点型, 这个根本就不是HDL语言的数据类型

2、可以被综合成电路

always、assign、define、for

while: 在有些工具是可以综合的, 但不建议使用。他和for一样, 循环的次数需要在编译之前就确定, 动态改变循环次数的语句是不可综合的。

initial: 虽然不能综合, 但并不是不能用。

initial 是不可综合, 但是可以赋初值 (初始化寄存器的值), 没有行为描述。

说的不可综合是指的不能生成实际的电路，不是说设计里面写了 initial 会报错 或者被忽略。

这里initial之所以不能综合，是因为它只能执行一次，而always会根据时钟条件无限次触发，所以always里面语句就像硬件电路，来一个时钟，执行一次。而initial就像是初始化程序，最开始把变量初始化，执行一次后不能再执行，直到再次复位。

可综合、不可综合的总结

可综合电路：

行为级可综合语法和数据流级语法合在一起被称为RTL级，该级别的模型是可以被综合成电路进而实现的。

- 1、module 和endmodule 作为模块声明的关键字，必然是可以被综合的。
- 2、输入input，输出output和双向端口inout的声明是可以被综合的。
- 3、变量类型reg、wire、integer都是可以被综合的。
- 4、参数parameter和宏定义define是可以被综合的。
- 5、所有的Verilog HDL 内建门都是可以使用的。
- 6、数据流级assign语句是可以被综合的。
- 7、行为级中敏感列表支持电平和边沿变化，类似posedge、negedge都是可以被综合的。
- 8、always、function是可被综合的，task中若不含延迟也是可被综合的。
- 9、顺序块begin...end可以被综合
- 10、if 和case语句可被综合

不可综合电路：

- 1、初始话initial结构不可被综合，电路中不会存在这样的单元。电路中一旦通电就会自动获得初始值，除此以外时序电路可以用复位端完成初始化组合电路不需要初始化。
- 2、#带来的延迟不可综合，电路中同样不会出现这样简单的延迟电路，所有的延迟电路都要通过计时电路或交互信号来完成
- 3、并行块fork...join不可综合，并行块的语义在电路中不能被转化。
- 4、用户自定义原语UP不可被综合
- 5、时间变量time和实数变量real不能被综合
- 6、wait、event、repeat、forever等行为级语法不可被综合。
- 7、一部分操作符可能不会被综合，例如除法/操作和求余%操作

3、x、z

寄存器未做赋值操作的话，其值是未知，在Verilog中用x表示；

线网类型可以理解为导线，其值是高阻，在Verilog中用z表示。

4、时间尺度：timescales

`timescale<时间单位>/<时间精度>

`timescale 100ns/10ns

#2.54 //延迟250ns

#2.56 //延迟260ns

对于第一个2.54个延迟单位，暂时不看精度的化是 $2.54 \times 100\text{ns} = 254\text{ns}$ ，但是这里精度是10ns，所以这里的实际延迟应该是250ns；相应的，对于第二个延迟则是260ns

单位就决定了你写“#xx”的时候，xx乘以多少。

而精度则决定了，当“xx乘以单位”出现比精度位小的情况时，

比如精度为10ns，但出现了4ns/5ns这样的情况

5、task、function

关于任务和函数的区别，下列说法错误的是(A)。（华为硬件逻辑实习岗）

- A 函数可以包含时延和时序控制
- B 函数不能调用任务
- C 函数必须带有至少一个输入
- D 函数只能返回一个值

函数不能含有延迟，A错误。

函数可以调用其他函数，但不能调用任务，B正确。

函数至少有一个输入变量，C正确。

函数只有一个返回值，不能声明其他输出端口，D正确

Verilog HDL中通过task和function关键字来声明任务和函数。利用任务和函数可以把一个很大的程序模块分解成许多较小的任务和函数便于理解和调试。

任务

通过task和endtask对任务进行声明。如果子程序满足以下任一条件，则公共子程序的描述必须使用任务而不能使用函数：

1. 子程序中包含延迟、时序或者时间控制结构。
2. 没有输出或者输出变量的数量大于1。
3. 没有输入变量。

任务结构

task <任务名>;

端口及数据类型声明语句;

局部变量声明语句

begin

语句1;

语句2;

.....

语句n;

end

endtask

任务可以调用其他任务或函数，也可以调用该任务本身。

不允许出现过程块（initial或always过程块）。

任务的调用是通过任务调用语句实现的。任务调用语句列出了传入任务和传出任务的参数值。

任务的调用格式如下：

<任务名>(端口1, 端口2, ..., 端口n);

任务调用语句只能出现在过程块内。

函数

Verilog HDL中的函数通过关键字function和endfunction对函数进行声明。对于一个子程序来说，如果下面的所有条件全部成立，则可以使用函数来完成。

1. 子程序内不含有延迟、时序或者时间控制结构。
2. 子程序只有一个返回值。
3. 至少有一个输入变量。
4. 没有输出或双向变量。
5. 不含有非阻塞赋值语句。

函数结构

function <返回值的类型或范围> <函数名>;

输入参量与类型声明语句;

局部变量声明语句;

begin

语句1;

语句2;

.....

语句n;

end

endfunction

函数名还是函数内部的一个变量，

函数调用后的返回值就是通过这个变量传递给调用语句的。

函数内可以对其他函数进行调用，但是函数不能调用其他任务。

返回值的类型缺省情况下是reg类型

不能有输出端口声明

函数的调用格式如下：

<函数名>(<输入表达式1>, <输入表达式2>, ..., <输入表达式n>);

函数的调用不能单独作为一条语句出现，它只能作为一个操作数出现在调用语句内

即能出现在过程块中，也能出现在assign连续赋值语句中

【卷三：数电相关内容】

1、不同进制间的转换

1.1 十进制转二进制

1.1.1 正整数转二进制

正整数转二进制：除二取余，然后倒序排列，高位补零

1. $(46)_{10} \rightarrow$

$$\begin{array}{r} 2 \overline{)46} \\ 2 \overline{)23} \\ 2 \overline{)11} \\ 2 \overline{)5} \\ 2 \overline{)2} \\ 1 \end{array}$$

0
1
1
0
1



101110

8位 字长
高位补零.

00101110

1.1.1 负数转二进制

先将对应的正整数转换成二进制后，对二进制取反，然后对结果再加1

$$\begin{aligned} 2. \quad (-42)_{10} &\rightarrow \\ (42)_{10} &\rightarrow (00101110)_2 \\ &\quad \downarrow \text{按位取反} \\ &\quad 11010001 \\ &\quad \downarrow \text{再加1} \\ &\quad 11010010 \end{aligned}$$

1.1.1 小数转二进制

对小数点以后的数乘以2，取整，剩余小数再乘以2，直至小数部分为0（有可能永远有小数，此时按需停止）

注：小数部分和整数部分互不影响，分开转换，最后合在一起即可

$$\begin{aligned} 3. \quad (0.25)_{10} &\rightarrow \\ 0.25 \times 2 &= 0.5 \quad \rightarrow 0 \\ 0.5 \times 2 &= 1.0 \quad \rightarrow 1 \\ (0.25)_{10} &\rightarrow 0.01 \end{aligned}$$

1.2 二进制转十进制

1.2.1 整数二进制转十进制

高位补零（比如数据为8位），再确定是有符号还是无符号，若是有符号整数，当最高位是0为正数，1为负数；

- 正整数，去除符号位，各个位数与对应2次幂相乘，最后求和。
- 负整数，去除符号位，减一，取反，再换算，记得添加整数的负号

$$1. (11010010)_2 \rightarrow$$

有符号位. 是负数

去除符号位, 再减1, 按位取反.

$$1010010 - 1 = 1010001$$

$$\text{取反} \rightarrow 0101110 \Rightarrow \begin{array}{cccccc} 1 & 0 & 1 & 1 & 1 & 0 \\ 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \end{array}$$

$$\text{对应2次幂乘积求和} \quad 32 + 8 + 4 + 2 = 46$$

1.2.2 小数二进制转十进制

各个位数与对应2的负次幂相乘, 最后求和。

$$2. (0.01)_2 \rightarrow$$

$$\begin{array}{ccc} 0 & . & 0 & 1 \\ 2^0 & & 2^{-1} & 2^{-2} \end{array} \quad \text{对应2的负次幂}$$

$$2^{-2} = \frac{1}{2^2} = \frac{1}{4} = 0.25$$

2、原码、反码、补码

3、数据宽度问题

N位二进制数所能表示的数据范围

有符号数（补码）： $-2^{(N-1)} \sim 2^{(N-1)}-1$

***正数范围-1，是因为0算在正数范围中**

如， $N = 8$ ，则表示范围是： $-128 \sim 127$ 。

无符号数： $0 \sim 2^N-1$

如， $N = 8$ ，则表示范围是： $0 \sim 255$ 。

有符号定点数：

如 $N = 8$, 3Q5：意思为共8位，3位整数位5位小数位。

如果有符号，那么整数位被占去一位。

所以整数部分其实只有2位，最大为4。

小数位有5位，那么最大值为：

$$4 + \sum 2^{(-i)} \quad (i = 1 \dots 5) - 2^{(-5)} ;$$

整数 + 所有小数位为1时的值 - 精度（最小的最小值）

为什么最大值要减1个精度，同样是因为0占掉了一个范围。

$$\text{最小值则为 } -4 - \sum 2^{(-i)} \quad (i = 1 \dots 5)$$

N bit数和M bit数相加、相乘后需要多少bit?

相加相乘后需要的数据位宽，若无已知数据范围，按照基本规律：

相加位宽+1，相乘位宽为N+M（保证宽度足够）

4、卡诺图化简

要注意以下几点：

- 1、包围圈越大越好（合并后得到的乘积项中因子最少）。
- 2、包围圈的个数越少越好（表达式乘积项项数最少）。
- 3、同一个“1”方块可以被圈多次（因为 $A+A=A$ ，对逻辑函数的正确性无影响）。
- 4、每个圈要有新的成分，如果某一圈中所有的“1”方块均被别的包围圈包围，则此圈所表示的乘积项是多余的。

- 5、画包围圈时，可先圈大，后圈小。
- 6、不要遗漏任何方块（否则函数会出错）

5、单目、双目、逻辑与或、按位与或

单目：

- 当“&”作为单目操作符时，是“缩位与”

双目：

- 当“&”作为双目操作符时，是“按位与”

“!”表示逻辑取反

“~”表示按位取反

6、标识符要区分大小写

COUNT与count是不同的标识符名称

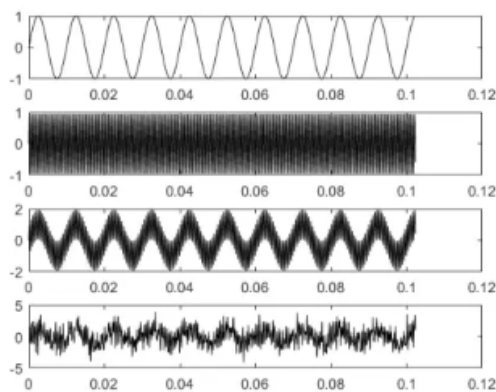
【卷四：项目相关内容】

1、快速傅里叶变换 FFT

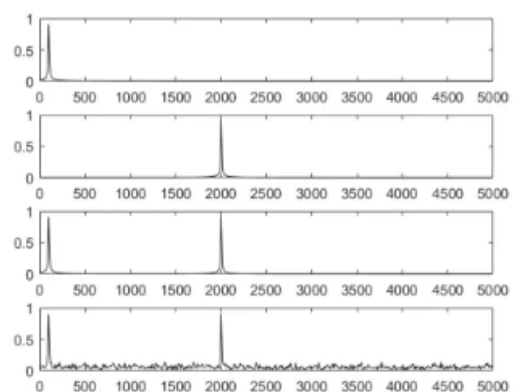
作用：把波形（时域信号）变换到频域，便于分析一些在时域里不容易看到的特性。

输入：N点时域信号

输出：N点频域信号（复数），每一个点就对应着一个频率点。这个点的模值，就是该频率值下的幅度特性。



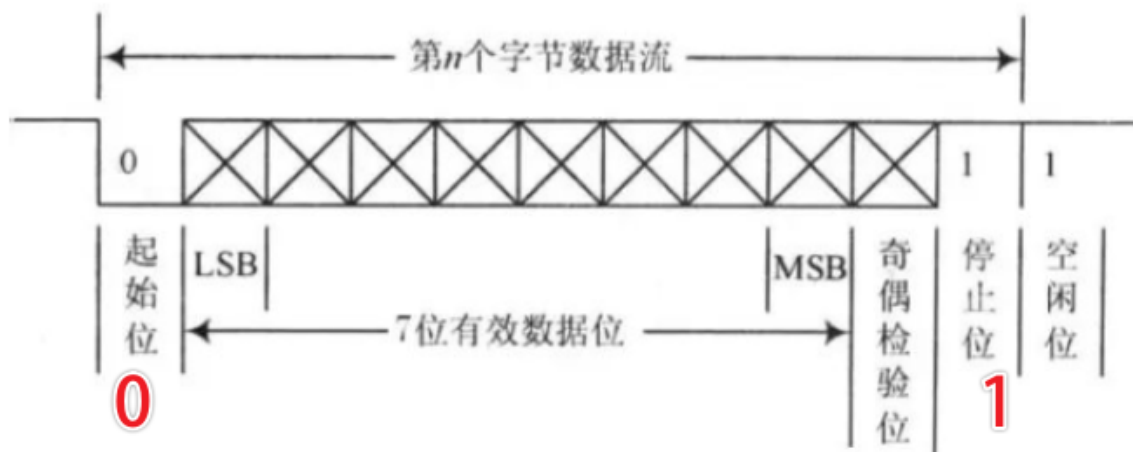
时域：正弦信号的叠加



频域：明显看到叠加信号

【卷五：协议、总线】

1、UART



如何发送数据

并行数据转串行数据，然后将串行数据发出去

如何接收数据

将接收到的串行数据转为并行数据

以什么样的速度收发数据

虽然接收端和发送端是异步的，即不需要使用同一个时钟，但接收端和发送端要以**相同的波特率**进行传输数据

由于UART是异步的，因此不需要使用时钟线，只需要2根信号线进行数据传输

波特率

波特率:Tx Rx必须相同 单位：/bps 每秒钟传输多少个比特位

常见的波特率有：9600，19200，38400，57600，**115200**

接收发送端的时钟频率不需要一样，但是波特率需要一致

50MHz发送频率： $1/115200 \times 50\text{MHz}$ 个时钟周期传1bit

20MHz接收频率： $1/115200 \times 20\text{MHz}$ 个时钟周期收1bit

概念：1秒钟能传多少个bit数据

计算： $50\text{MHz}/115200 = 50 \times 10^6 / 115200 =$ 花几个时钟周期才能传输1bit数据

时序图

将输入数据打2拍的原因：

FPGA接收模块时快时钟，PC端的时钟是慢时钟，因此从慢时钟同步到快时钟域，打2拍消除亚稳态

由于打了第一拍的结果还是不太稳定，因此需要再打一拍，用第2、3拍来检测下降沿

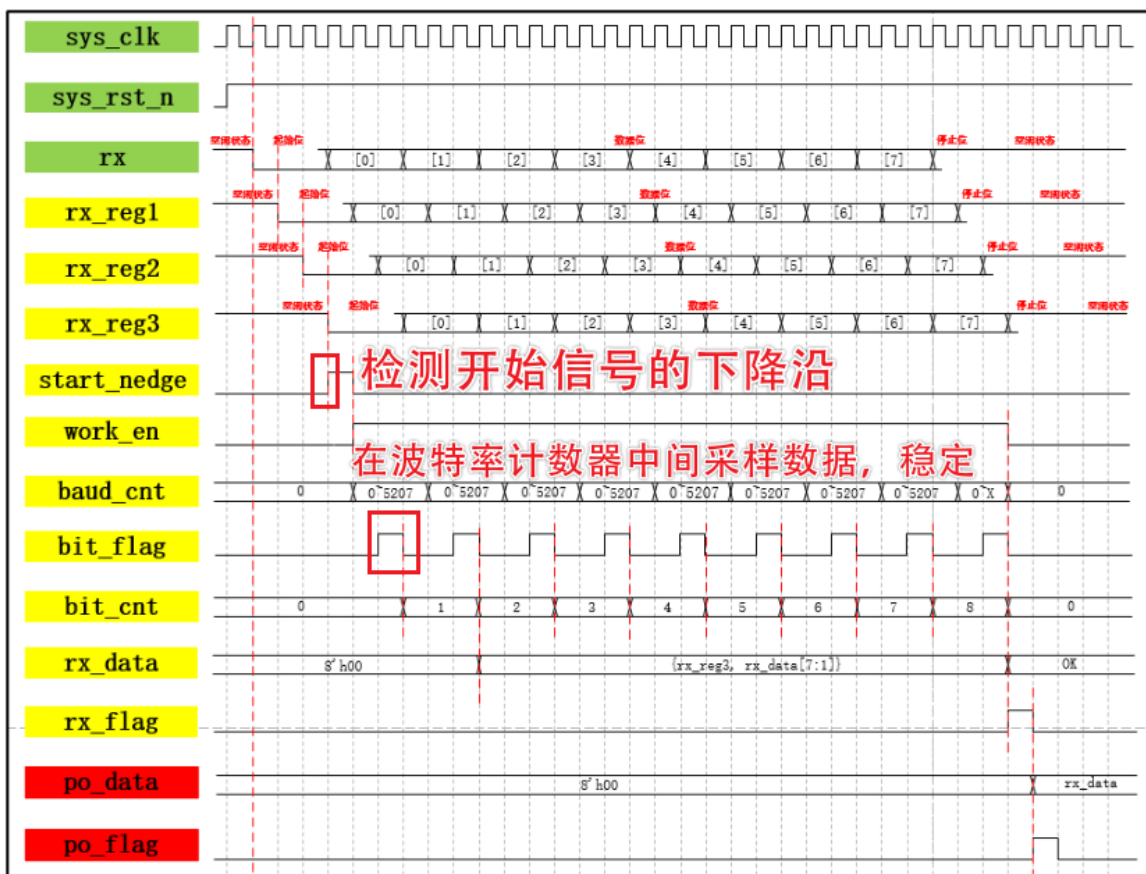


图 28-11 串口接收模块波形图

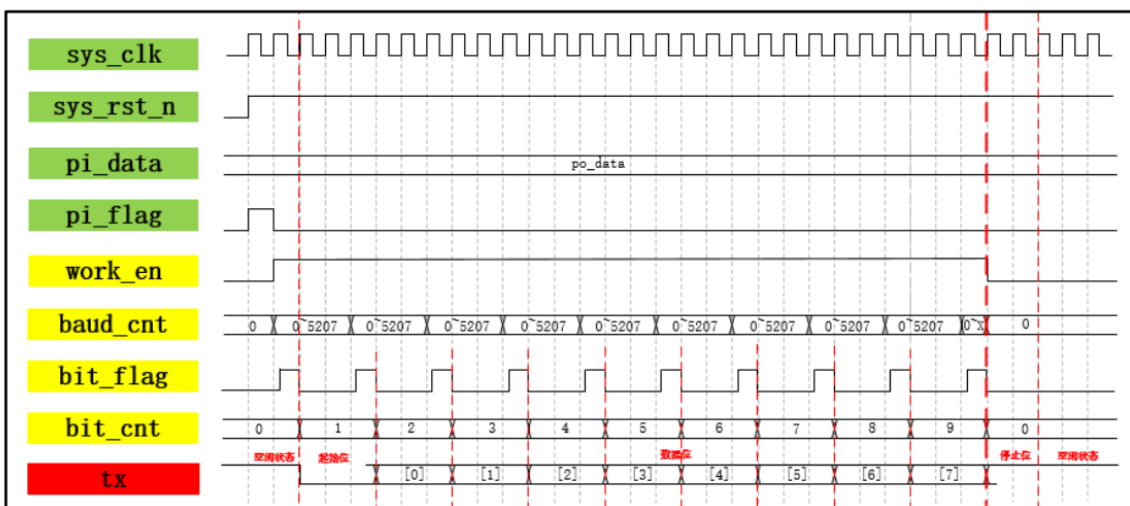
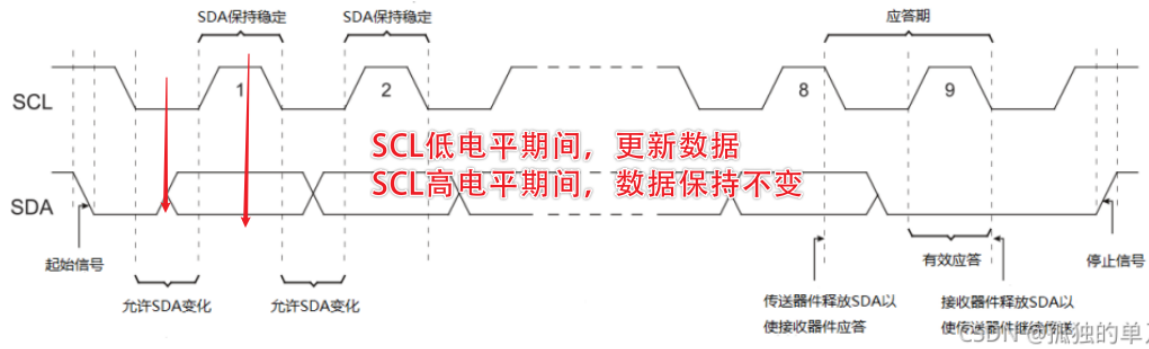


图 28-31 串口发送模块波形图

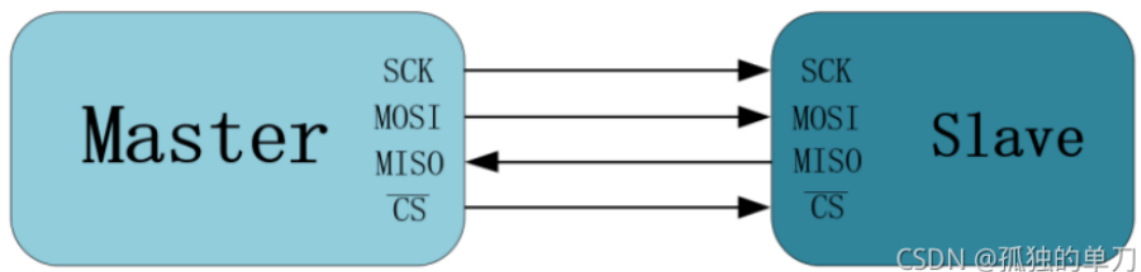
2、I2C

一个 I2C 总线只使用两条总线线路，一条双向串行数据线(SDA)，一条串行时钟线 (SCL)。数据线即用来表示数据，时钟线用于数据收发同步

每个连接到总线的设备都有一个独立的地址，主机可以利用这个地址进行不同设备之间的访问



3、SPI



结构：主机、从机

工作模式:

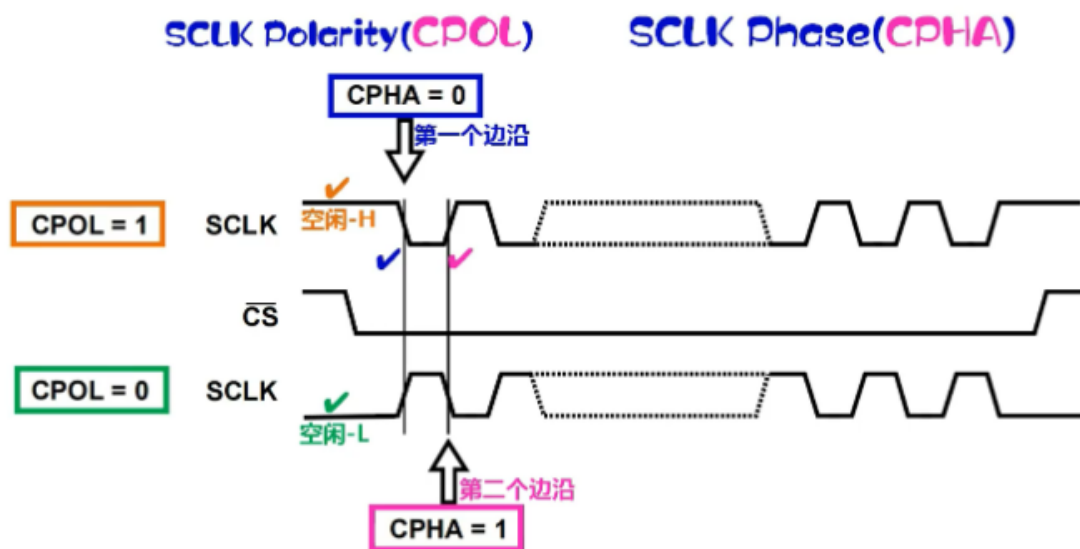
CPOL:

- 0: 总线空闲时, 时钟SCLK为低电平
- 1: 总线空闲时, 时钟SCLK为高电平

CPHA:

- 0: 时钟第1个边沿对数据进行采样
- 1: 时钟第2个边沿对数据进行采样

SPI工作模式



CPOL=0、CPHA=0

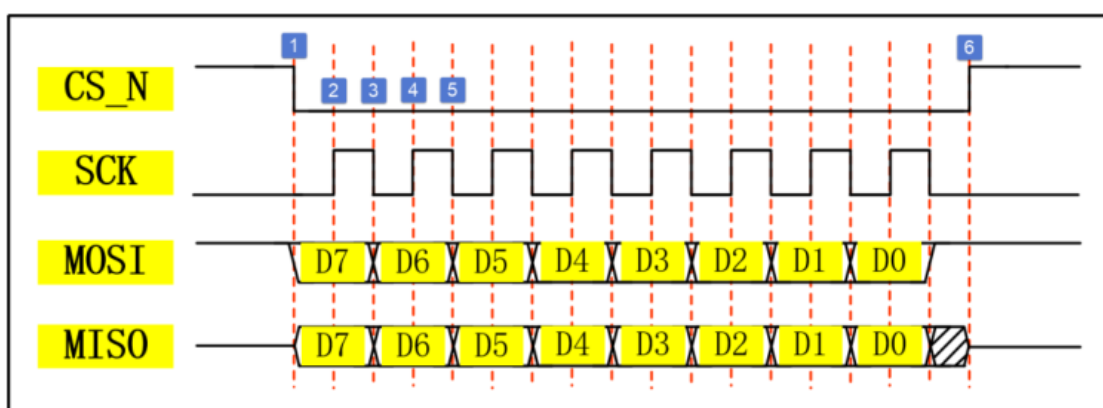


图 38-6 SPI 模式 0 通讯时序图

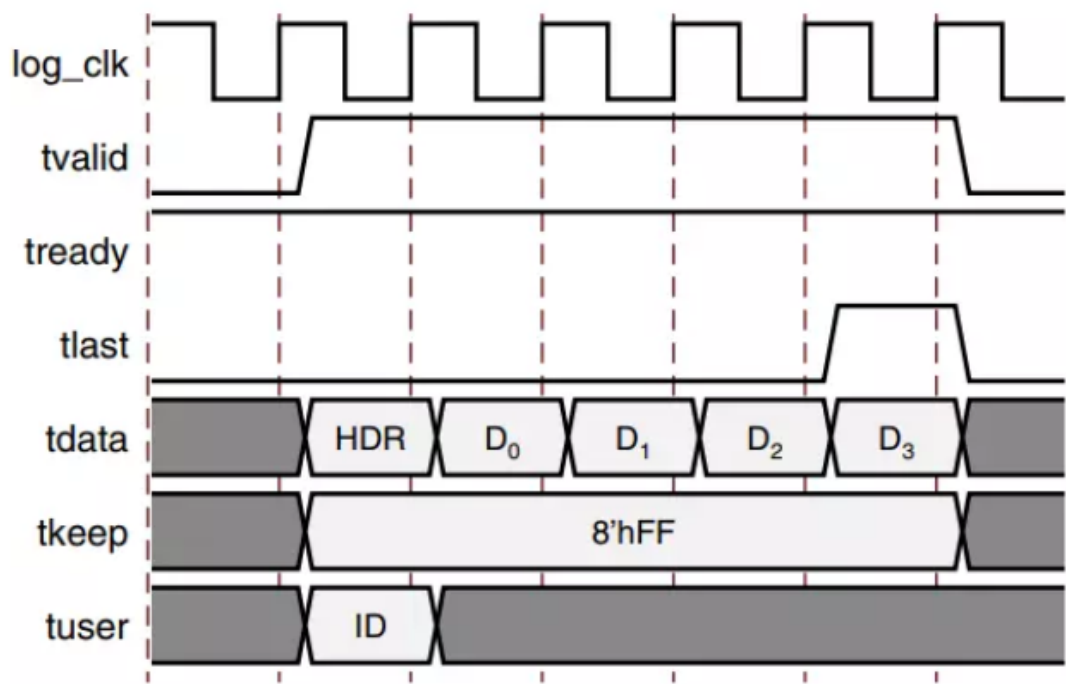
SPI总线传输只需要4根线就能完成, 这四根线的作用分别如下:

- SCK (Serial Clock): 时钟信号线, 用于同步通讯数据。由通讯主机产生, 决定了通讯的速率, 不同的设备支持的最高时钟频率不同

- MOSI (Master Output, Slave Input): 主设备输出/从设备输入引脚。主机的数据从这条信号线输出，从机由这条信号线读入主机发送的数据，数据方向由主机到从机
- MISO (Master Input, Slave Output): 主设备输入/从设备输出引脚。主机从这条信号线读入数据，从机的数据由这条信号线输出到主机，数据方向由从机到主机
- CS (Chip Select): 片选信号线。当有多个 SPI 从设备与 SPI 主机相连时，设备的其它信号线 SCK、MOSI 及 MISO 同时并联到相同的 SPI 总线上，即无论有多少个从设备，都共同使用这 3 条总线；而每个从设备都有独立的片选信号线，即有多少个从设备，就有多少条片选信号线。相当于由 SPI 构成的通信系统中，通过 CS 片选信号来决定通信的从设备是哪一台。通信期间低电平有效，表示对应从机被选中

4、AXI-stream

握手：主机发送的tvalid，和从机发送的tready都拉高，握手成功，可以将主机数据传输给从机

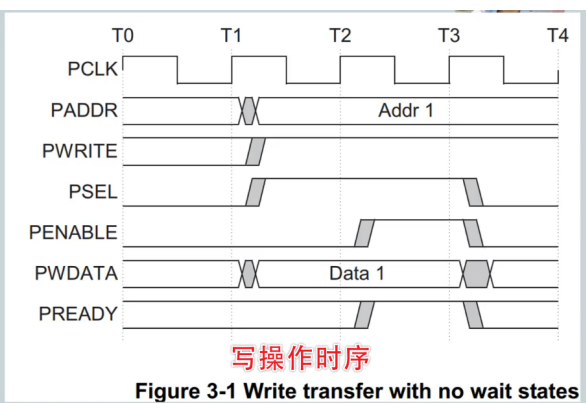
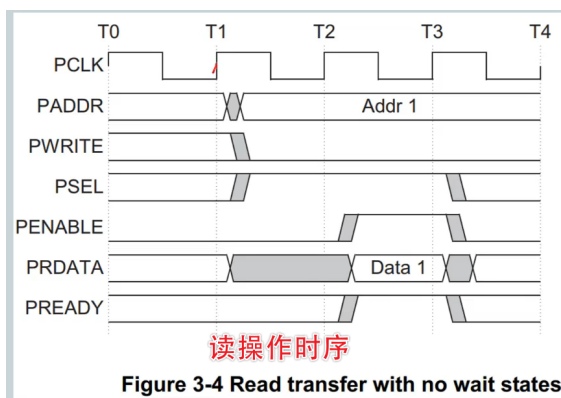


5、APB总线

接口定义：

Signal	Description
PCLK	时钟。APB协议里所有的数据传输都在PCLK上升沿进行
PRESETn	复位。低电平有效
PADDR	APB地址总线。最大宽度为32位
PSELx	选通。APB master会将此信号生成给每个slave。它指示已选择的slave，并且需要进行数据传输。每个slave都有一个PSELx信号。
PENABLE	使能。当它为高时，表示数据有效
PWRITE	读写控制。为高时表示写操作，为低时表示读操作
PWDATA	写数据。master通过PWDATA将数据写到slave，该总线最大宽度为32位
PRDATA	读数据。master通过PRDATA将数据从slave读取回来，该总线最大宽度为32位

看了这个端口定义，大家可能对master和slave还不太明白，解释一下，可以把APB看成一种“线”，这种“线”是用来连接两个设备的，一个就是master，一个就是slave，**所有的数据传输都是master来控制的，slave来回应。**



通过读写操作的时序图，我们可以看到，无论是读还是写，都是**两个周期**。

第一个周期：PSEL为高，PENABLE为低，这个时候为data的传输做准备工作，包括准备数据、地址、读写控制信号；
第二个周期：PSEL和PENABLE同时为高，进行data的传输。

【卷六：职业相关】

前端的越了解算法、协议和系统越值钱，后端的越了解工具和方法学越值钱。

verilog不是英语，你每写一句脑子里就应该有一个电路

FPGA工程师的工作内容

FPGA工程师每天干的活主要是画图，画时序图，画框图，画关系图，用软件画图分析，用代码画图，用手画图，用文字画图。然后就是看图分析错误，看图写话。看图算参数，看图定协议